Vellore Institute of Technology (VIT)

School of Computer Science and Engineering

Machine Learning Lab (MACSE504) - Mini-Project Phase 1 (Part A)

- **Student Name:** Vedant
- **Register Number:** 26MML0045
- **Assigned Question:** Question 3: Used Car Valuation & Elasticity Analysis using Regularized Regression
- **Component:** Part A: Exploratory Data Analysis & Preprocessing

---

Project Overview & Objectives

This project focuses on building an automated valuation model for a used car marketplace. Resale prices are estimated based on characteristics like age, mileage, fuel type, transmission, and dealer type. Estimating prices accurately is crucial to avoid underpricing listings (loss of revenue) or overpricing them (which increases inventory holding time).

We are using the CarDekho dataset (`Car details v3.csv`). In Part A, the objectives are:

1. **Data Cleaning & Feature Extraction:** Calculate vehicle age, clean string-encoded values in columns like `mileage` (kmpl or km/kg), `engine` (CC), and `max_power` (bhp), and impute missing values using brand-level statistics.
2. **Target Transformation:** Analyze the distribution of `selling_price` and apply a log-transformation to reduce skewness and stabilize variance.
3. **Data Visualization:** Explore price trends against continuous variables like kilometers driven (`km_driven`) and categorical variables like `fuel` using scatter plots, boxplots, and correlation matrices.
4. **Interactive Dashboard:** Set up a simple interactive widget to filter the data and inspect these relationships dynamically.

```
# Standard library & numerical computing
import os
import sys
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import scipy.stats as stats

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Interactive Widgets for GUI Interface (Rubric Requirement)
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

# Configuration for aesthetic, publication-ready figures
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams.update({
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'legend.fontsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 120,
    'axes.edgecolor': '#cccccc',
    'axes.linewidth': 0.8
})

# Custom palette for automotive fuel types
FUEL_PALETTE = {
    'Diesel': '#1f77b4',
    'Petrol': '#ff7f0e',
    'CNG': '#2ca02c',
    'LPG': '#d62728'
}

print("Libraries successfully imported.")
print(f"Pandas Version: {pd.__version__} | NumPy Version: {np.__version__} | Matplotlib Version: {plt.matplotlib.__version_
```

```
Libraries successfully imported.
Pandas Version: 2.3.3 | NumPy Version: 1.26.4 | Matplotlib Version: 3.10.0
```

1. Feature Extraction & Data Preprocessing

Objectives:

- Load the dataset and inspect its structure.
- Calculate the current vehicle age using the manufacturing year.
- Convert technical specs (`mileage`, `engine`, `max_power`) from text strings to numeric values.
- Handle missing values and replace physically impossible data (like 0 mileage) using group-wise medians.

```
# Rubric Requirement: Interactive interface to load dataset and inspect properties
DATASET_PATH = 'Car details v3.csv'

def load_car_dataset(file_path):
    """Loads CarDekho dataset, checks file existence and returns raw DataFrame."""
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"Dataset file not found at: {file_path}")
    df = pd.read_csv(file_path)
    return df

# Load primary dataset
df_raw = load_car_dataset(DATASET_PATH)
df = df_raw.copy()

print(f"Dataset Successfully Loaded: {DATASET_PATH}")
print(f"Total Rows (Used Car Listings): {df.shape[0]:,}")
print(f"Total Attributes: {df.shape[1]}")
print("\nInitial Column Data Types and Non-Null Counts:")
print(df.info())
print("\nFirst 5 Records (Raw):")
display(df.head())
```

```
Dataset Successfully Loaded: Car details v3.csv
Total Rows (Used Car Listings): 8,128
Total Attributes: 13

Initial Column Data Types and Non-Null Counts:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8128 entries, 0 to 8127
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   name           8128 non-null   object
 1   year           8128 non-null   int64
 2   selling_price  8128 non-null   int64
 3   km_driven      8128 non-null   int64
 4   fuel           8128 non-null   object
 5   seller_type    8128 non-null   object
 6   transmission   8128 non-null   object
 7   owner          8128 non-null   object
 8   mileage        7907 non-null   object
 9   engine         7907 non-null   object
 10  max_power      7913 non-null   object
 11  torque         7906 non-null   object
 12  seats          7907 non-null   float64
dtypes: float64(1), int64(3), object(9)
memory usage: 825.6+ KB
None

First 5 Records (Raw):
```

	name	year	selling_price	km_driven	fuel	seller_type	transmission	owner	mileage	engine	max_power	torque
0	Maruti Swift Dzire VDI	2014	450000	145500	Diesel	Individual	Manual	First Owner	23.4 kmpl	1248 CC	74 bhp	190Nm@ 2000rpm
1	Skoda Rapid 1.5 TDI Ambition	2014	370000	120000	Diesel	Individual	Manual	Second Owner	21.14 kmpl	1498 CC	103.52 bhp	250Nm@ 1500-2500rpm

Preprocessing Steps:

1. **Vehicle Age Calculation:** The dataset was collected in 2020. We will compute the vehicle age as:

$$\text{Vehicle Age} = 2020 - \text{year}$$

2. **String Cleaning:**
    - **Mileage:** Values are written as '23.4 kmpl' or '17.3 km/kg'. We will extract the numbers and convert them to floats. Additionally, any '0.0' mileage values will be set to NaN since operational cars cannot have zero mileage.
    - **Engine:** Text like '1248 CC' will be converted to numeric CC values.
    - **Max Power:** Text like '74 bhp' will be cleaned to represent numeric brake horsepower.
3. **Missing Value Imputation:** Rather than using global medians, we will impute missing values using the median of the specific car brand (derived from the first word of the car name). This ensures we preserve brand-specific specs (e.g., engine sizes).

```
# 1. Derive Vehicle Age
REFERENCE_YEAR = 2020
df['vehicle_age'] = REFERENCE_YEAR - df['year']
print("Vehicle Age Summary (in years):")
print(df['vehicle_age'].describe())

# 2. Extract Brand for Grouped Imputation
df['brand'] = df['name'].apply(lambda x: str(x).split()[0].strip())

# 3. Clean 'mileage' Feature
# Extract numeric floating value from 'xx.xx kmpl' or 'xx.xx km/kg'
df['mileage_val'] = df['mileage'].str.extract(r'([0-9.]+)').astype(float)
# Handle zero mileage anomaly (17 records with 0.0 kmpl)
zero_mileage_count = (df['mileage_val'] == 0).sum()
print(f"\nDetected {zero_mileage_count} physically impossible zero mileage entries. Mapping to NaN...")
df.loc[df['mileage_val'] == 0, 'mileage_val'] = np.nan

# 4. Clean 'engine' Feature
# Extract displacement in CC from 'xxxx CC'
df['engine_val'] = df['engine'].str.extract(r'([0-9.]+)').astype(float)

# 5. Clean 'max_power' Feature
# Extract horsepower from 'xx.xx bhp'
df['max_power_val'] = df['max_power'].str.extract(r'([0-9.]+)').astype(float)
df.loc[df['max_power_val'] == 0, 'max_power_val'] = np.nan

# 6. Check Missing Values Before Imputation
missing_before = df[['mileage_val', 'engine_val', 'max_power_val', 'seats']].isnull().sum()
print("\nMissing Values Count Before Imputation:")
print(missing_before)

# 7. Apply Brand-Conditioned Median Imputation
clean_cols = ['mileage_val', 'engine_val', 'max_power_val', 'seats']
for col in clean_cols:
    # First impute by brand median
    df[col] = df.groupby('brand')[col].transform(lambda s: s.fillna(s.median()))
    # Fallback to global median for brands with all-missing values
    df[col] = df[col].fillna(df[col].median())

# 8. Post-Imputation Verification
missing_after = df[['mileage_val', 'engine_val', 'max_power_val', 'seats']].isnull().sum()
print("\nMissing Values Count After Brand-Conditioned Imputation:")
print(missing_after)

# Display sample of preprocessed attributes
display(df[['name', 'year', 'vehicle_age', 'mileage_val', 'engine_val', 'max_power_val', 'selling_price', 'fuel']].head())
```

```
Vehicle Age Summary (in years):
count    8128.000000
mean        6.195989
std         4.044249
min         0.000000
25%         3.000000
50%         5.000000
75%         9.000000
max        37.000000
Name: vehicle_age, dtype: float64

Detected 17 physically impossible zero mileage entries. Mapping to NaN...

Missing Values Count Before Imputation:
mileage_val      238
engine_val       221
max_power_val    222
seats            221
dtype: int64

Missing Values Count After Brand-Conditioned Imputation:
mileage_val      0
engine_val       0
max_power_val    0
seats            0
dtype: int64
```

	name	year	vehicle_age	mileage_val	engine_val	max_power_val	selling_price	fuel
0	Maruti Swift Dzire VDI	2014	6	23.40	1248.0	74.00	450000	Diesel
1	Skoda Rapid 1.5 TDI Ambition	2014	6	21.14	1498.0	103.52	370000	Diesel
2	Honda City 2017-2020 EXi	2006	14	17.70	1497.0	78.00	158000	Petrol
3	Hyundai i20 Sportz Diesel	2010	10	23.00	1396.0	90.00	225000	Diesel
4	Maruti Swift VXI BSIII	2007	13	16.10	1298.0	88.20	130000	Petrol

2. Price Distribution & Log Transformation

Objectives:

- Examine the distribution of the target variable (`selling_price`).
- Apply a log transformation to handle high skewness and stabilize variance.
- Compare the distribution before and after using histograms and Q-Q plots.

```
# 1. Descriptive Statistics of Raw Selling Price
raw_price = df['selling_price']
raw_mean = raw_price.mean()
raw_median = raw_price.median()
raw_std = raw_price.std()
raw_skew = raw_price.skew()
raw_kurt = raw_price.kurtosis()

print("="*75)
print("RAW SELLING PRICE DISTRIBUTIONAL PROFILE:")
print(f"  - Count              : {len(raw_price):,}")
print(f"  - Mean Price         : ₹{raw_mean:,.2f}")
print(f"  - Median Price       : ₹{raw_median:,.2f}")
print(f"  - Standard Deviation : ₹{raw_std:,.2f}")
print(f"  - Min Price          : ₹{raw_price.min():,}")
print(f"  - Max Price          : ₹{raw_price.max():,}")
print(f"  - Skewness           : {raw_skew:.4f} (Severely Right-Skewed: > +1.0)")
print(f"  - Kurtosis           : {raw_kurt:.4f} (Leptokurtic / Heavy-Tailed: >> 3.0)")
print("="*75)

# 2. Apply Natural Log-Transformation
df['log_selling_price'] = np.log(df['selling_price'])
log_price = df['log_selling_price']

log_mean = log_price.mean()
log_median = log_price.median()
log_std = log_price.std()
log_skew = log_price.skew()
log_kurt = log_price.kurtosis()

print("\nLOG-TRANSFORMED SELLING PRICE PROFILE:")
print(f"  - Mean Log Price     : {log_mean:.4f}")
print(f"  - Median Log Price   : {log_median:.4f}")
print(f"  - Std Log Price      : {log_std:.4f}")
print(f"  - Skewness           : {log_skew:.4f} (Near-Symmetric: [-0.5, +0.5])")
```

```
print(f"  - Kurtosis           : {log_kurt:.4f} (Mesokurtic / Normal-like)")
print(f"  -> Skewness Reduction : {abs(raw_skew - log_skew)/raw_skew * 100:.2f}% reduction!")
print("="*75)
```

```
===========================================================================
RAW SELLING PRICE DISTRIBUTIONAL PROFILE:
  - Count              : 8,128
  - Mean Price         : ₹638,271.81
  - Median Price       : ₹450,000.00
  - Standard Deviation : ₹806,253.40
  - Min Price          : ₹29,999
  - Max Price          : ₹10,000,000
  - Skewness           : 4.1935 (Severely Right-Skewed: > +1.0)
  - Kurtosis           : 21.0813 (Leptokurtic / Heavy-Tailed: >> 3.0)
===========================================================================

LOG-TRANSFORMED SELLING PRICE PROFILE:
  - Mean Log Price     : 12.9734
  - Median Log Price   : 13.0170
  - Std Log Price      : 0.8391
  - Skewness           : 0.2233 (Near-Symmetric: [-0.5, +0.5])
  - Kurtosis           : 0.9867 (Mesokurtic / Normal-like)
  -> Skewness Reduction : 94.68% reduction!
===========================================================================
```

```
# Visualizing Raw vs Log-Transformed Price Distributions
fig, axes = plt.subplots(2, 2, figsize=(16, 11))

# 1. Histogram & KDE: Raw Selling Price
sns.histplot(df['selling_price'], kde=True, color='#d9534f', bins=50, ax=axes[0, 0])
axes[0, 0].set_title(f'Raw Selling Price Distribution (Skewness = +{raw_skew:.2f})', fontweight='bold')
axes[0, 0].set_xlabel('Selling Price (₹ in Millions)')
axes[0, 0].set_ylabel('Frequency')
axes[0, 0].axvline(raw_mean, color='blue', linestyle='--', label=f'Mean (₹{raw_mean/1e5:.1f} Lakh)')
axes[0, 0].axvline(raw_median, color='black', linestyle='-', label=f'Median (₹{raw_median/1e5:.1f} Lakh)')
axes[0, 0].legend()

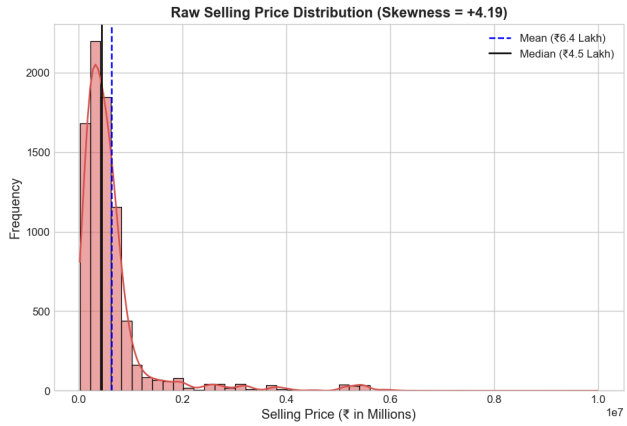
# 2. Histogram & KDE: Log-Transformed Selling Price
sns.histplot(df['log_selling_price'], kde=True, color='#2b5c8f', bins=50, ax=axes[0, 1])
axes[0, 1].set_title(f'Log-Transformed Selling Price Distribution (Skewness = +{log_skew:.2f})', fontweight='bold')
axes[0, 1].set_xlabel('ln(Selling Price)')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].axvline(log_mean, color='blue', linestyle='--', label=f'Mean ({log_mean:.2f})')
axes[0, 1].axvline(log_median, color='black', linestyle='-', label=f'Median ({log_median:.2f})')
axes[0, 1].legend()

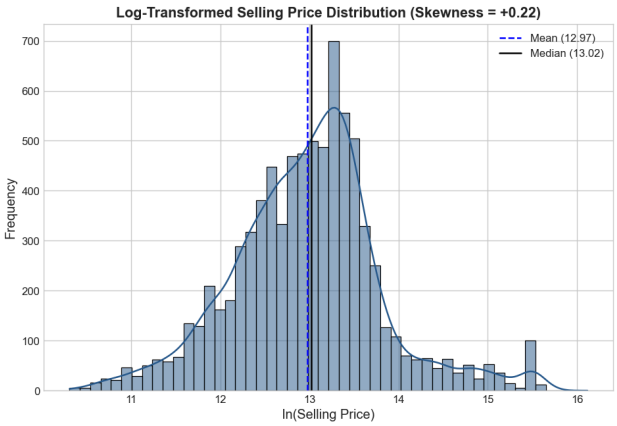
# 3. Normal Q-Q Plot: Raw Selling Price
stats.probplot(df['selling_price'], dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Normal Q-Q Plot: Raw Selling Price (Extreme Departure)', fontweight='bold')
axes[1, 0].get_lines()[0].set_color('#d9534f')
axes[1, 0].get_lines()[0].set_markersize(3)
axes[1, 0].get_lines()[1].set_color('black')

# 4. Normal Q-Q Plot: Log-Transformed Selling Price
stats.probplot(df['log_selling_price'], dist="norm", plot=axes[1, 1])
axes[1, 1].set_title('Normal Q-Q Plot: Log-Transformed Selling Price (Near-Linear Fit)', fontweight='bold')
axes[1, 1].get_lines()[0].set_color('#2b5c8f')
axes[1, 1].get_lines()[0].set_markersize(3)
axes[1, 1].get_lines()[1].set_color('black')

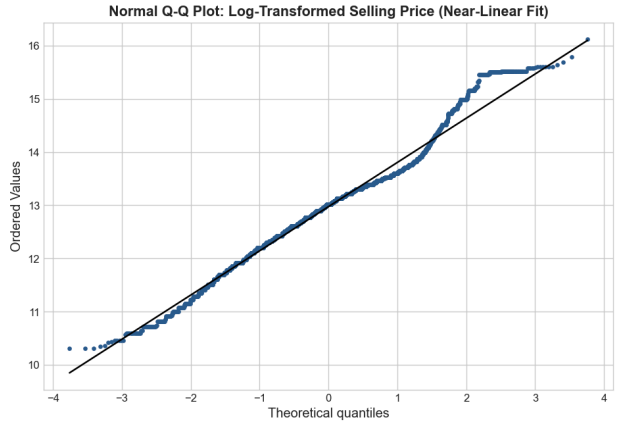
plt.tight_layout()
plt.show()

# Theoretical Summary:
print("ECONOMIC & STATISTICAL JUSTIFICATION FOR LOG-TRANSFORMATION:")
print("1. Raw selling prices follow a Pareto/Lognormal power-law distribution where high-value luxury cars create a heavy r
print("2. Linear regression models (OLS, Ridge, Lasso) assume normally distributed residuals and homoscedasticity (constant
print("3. In raw price models, predicting a ₹50 Lakh car with 10% error incurs a ₹5 Lakh residual penalty, dwarfing a ₹40,0
print("4. Under log-transformation, linear coefficients measure constant PERCENTAGE elasticities, stabilizing variance acro
```










```
ECONOMIC & STATISTICAL JUSTIFICATION FOR LOG-TRANSFORMATION:
1. Raw selling prices follow a Pareto/Lognormal power-law distribution where high-value luxury cars create a heavy right t
2. Linear regression models (OLS, Ridge, Lasso) assume normally distributed residuals and homoscedasticity (constant error
3. In raw price models, predicting a ₹50 Lakh car with 10% error incurs a ₹5 Lakh residual penalty, dwarfing a ₹40,000 err
4. Under log-transformation, linear coefficients measure constant PERCENTAGE elasticities, stabilizing variance across all
```

3. Visualizing Price Trends and Relationships

Objectives:

- Check how resale prices deprecate as kilometers driven increase.
- Examine price differences across different fuel types (Diesel, Petrol, CNG, LPG).
- Calculate Pearson and Spearman correlation matrices to assess relationships between variables.

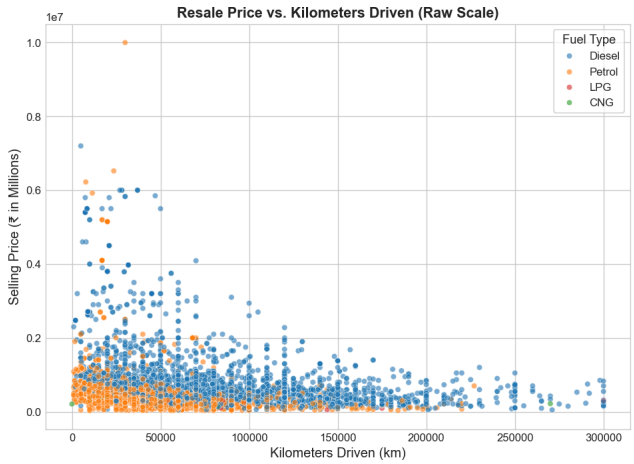
```
# 1. Price Trends vs Kilometers Driven
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

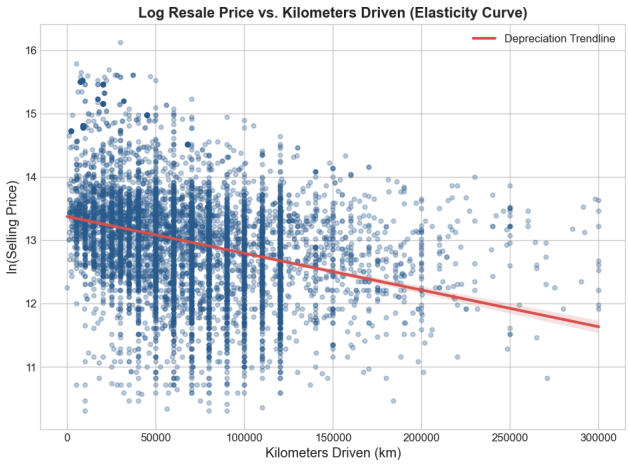
# Subplot A: Raw Price vs Km Driven (capped at 300,000 km for visual clarity)
sns.scatterplot(
    data=df[df['km_driven'] <= 300000],
    x='km_driven',
    y='selling_price',
    hue='fuel',
    palette=FUEL_PALETTE,
    alpha=0.6,
    s=25,
    ax=axes[0]
)
axes[0].set_title('Resale Price vs. Kilometers Driven (Raw Scale)', fontweight='bold')
axes[0].set_xlabel('Kilometers Driven (km)')
axes[0].set_ylabel('Selling Price (₹ in Millions)')
axes[0].legend(title='Fuel Type', frameon=True)

# Subplot B: Log Selling Price vs Km Driven with OLS Regression Trendline
sns.regplot(
    data=df[df['km_driven'] <= 300000],
    x='km_driven',
    y='log_selling_price',
    scatter_kws={'alpha': 0.3, 'color': '#2b5c8f', 's': 15},
    line_kws={'color': '#d9534f', 'linewidth': 2.5, 'label': 'Depreciation Trendline'},
    ax=axes[1]
)
axes[1].set_title('Log Resale Price vs. Kilometers Driven (Elasticity Curve)', fontweight='bold')
axes[1].set_xlabel('Kilometers Driven (km)')
axes[1].set_ylabel('ln(Selling Price)')
axes[1].legend()

plt.tight_layout()
plt.show()

# Compute correlation between mileage driven and price
corr_km_raw = df['km_driven'].corr(df['selling_price'])
corr_km_log = df['km_driven'].corr(df['log_selling_price'])
print(f"Pearson Correlation between km_driven and raw selling_price : {corr_km_raw:.4f}")
print(f"Pearson Correlation between km_driven and log_selling_price : {corr_km_log:.4f} (Stronger negative association)")
```






```
Pearson Correlation between km_driven and raw selling_price : -0.2255
Pearson Correlation between km_driven and log_selling_price : -0.2527 (Stronger negative association)
```

```
# 2. Resale Price Distribution Across Fuel Types
fuel_order = ['Diesel', 'Petrol', 'CNG', 'LPG']

# Summary table of price by fuel type
fuel_stats = df.groupby('fuel')['selling_price'].agg([
    'count',
    lambda x: x.median(),
    lambda x: x.mean(),
    lambda x: x.std()
]).rename(columns={'<lambda_0>': 'median', '<lambda_1>': 'mean', '<lambda_2>': 'std'}).loc[fuel_order]

print("Resale Price Summary by Fuel Type (in ₹):")
display(fuel_stats.style.format({'count': '{:,}', 'median': '₹{:,.0f}', 'mean': '₹{:,.0f}', 'std': '₹{:,.0f}'}))

# Visualizations: Boxplot and Violin Plot
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Subplot A: Boxplot of Log Selling Price by Fuel Type
sns.boxplot(
    data=df,
    x='fuel',
    y='log_selling_price',
    order=fuel_order,
    palette=FUEL_PALETTE,
    ax=axes[0],
    showmeans=True,
    meanprops={"marker":"o", "markerfacecolor":"white", "markeredgecolor":"black", "markersize":"8"}
)
axes[0].set_title('Log Resale Price Distribution across Fuel Types (Boxplot)', fontweight='bold')
axes[0].set_xlabel('Fuel Type')
axes[0].set_ylabel('ln(Selling Price)')

# Annotate median price on boxplots
medians_log = df.groupby('fuel')['log_selling_price'].median().loc[fuel_order]
for i, (fuel, med) in enumerate(medians_log.items()):
    axes[0].text(i, med + 0.15, f"{med:.2f}\n(₹{df[df['fuel']==fuel]['selling_price'].median()/1e5:.1f}L)",
                 ha='center', va='bottom', fontsize=10, fontweight='bold', color='#111111')

# Subplot B: Violin Plot showing Kernel Density by Fuel Type
sns.violinplot(
    data=df,
    x='fuel',
```

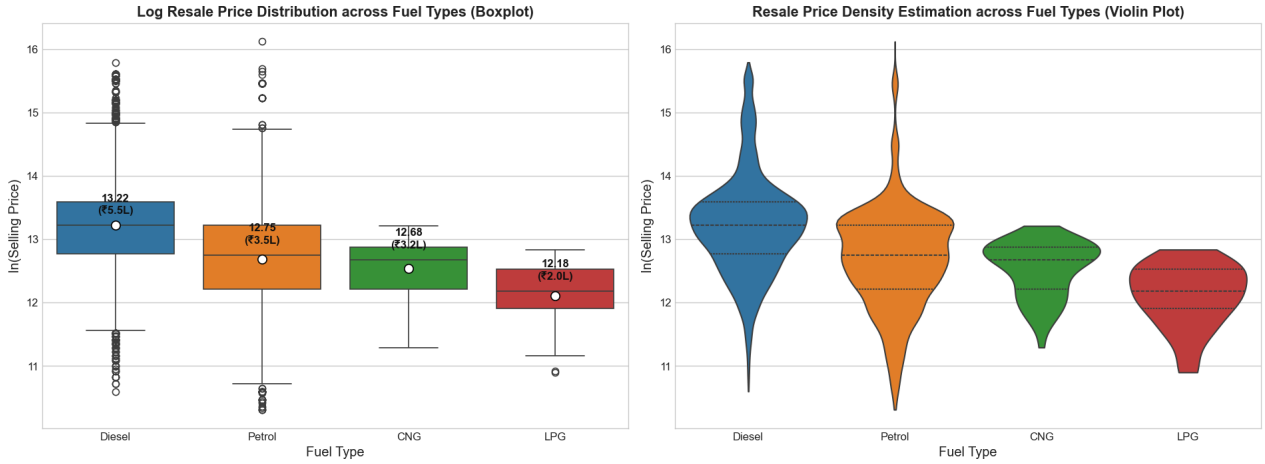
```
    y='log_selling_price',
    order=fuel_order,
    palette=FUEL_PALETTE,
    inner='quartile',
    cut=0,
    ax=axes[1]
)
axes[1].set_title('Resale Price Density Estimation across Fuel Types (Violin Plot)', fontweight='bold')
axes[1].set_xlabel('Fuel Type')
axes[1].set_ylabel('ln(Selling Price)')

plt.tight_layout()
plt.show()

# Domain finding
diesel_med = df[df['fuel']=='Diesel']['selling_price'].median()
petrol_med = df[df['fuel']=='Petrol']['selling_price'].median()
premium_pct = ((diesel_med - petrol_med) / petrol_med) * 100
print(f"\nKEY AUTOMOTIVE MARKET INFERENCE:")
print(f"Diesel vehicles command a substantial resale price premium: Median Diesel Price = ₹{diesel_med:,.0f} vs Petrol = ₹
print(f"Resale Value Premium of Diesel over Petrol: +{premium_pct:.1f}%!")
```

```
Resale Price Summary by Fuel Type (in ₹):
```

fuel	count	median	mean	std
Diesel	4,402	₹550,000	₹791,453	₹908,895
Petrol	3,631	₹345,000	₹462,441	₹624,789
CNG	57	₹320,000	₹301,017	₹111,582
LPG	38	₹195,500	₹200,421	₹84,921




```
KEY AUTOMOTIVE MARKET INFERENCE:
Diesel vehicles command a substantial resale price premium: Median Diesel Price = ₹550,000 vs Petrol = ₹345,000.
Resale Value Premium of Diesel over Petrol: +59.4%!
```

```
# 3. Correlation Matrix for Continuous Engineered & Technical Attributes
corr_features = ['selling_price', 'log_selling_price', 'vehicle_age', 'km_driven', 'mileage_val', 'engine_val', 'max_power_
corr_matrix_pearson = df[corr_features].corr(method='pearson')
corr_matrix_spearman = df[corr_features].corr(method='spearman')

fig, axes = plt.subplots(1, 2, figsize=(16, 6.5))

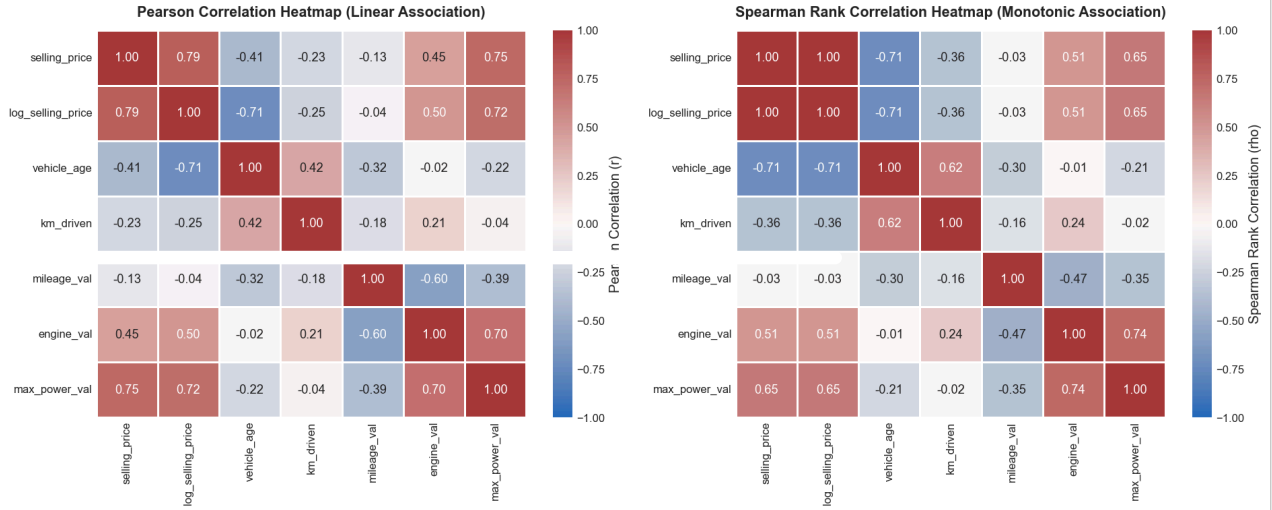
# Pearson Heatmap
sns.heatmap(corr_matrix_pearson, annot=True, fmt=".2f", cmap='vlag', vmin=-1, vmax=1,
            cbar_kws={'label': 'Pearson Correlation (r)'}, ax=axes[0], linewidths=0.8)
```

```
axes[0].set_title('Pearson Correlation Heatmap (Linear Association)', fontweight='bold', pad=12)

# Spearman Heatmap
sns.heatmap(corr_matrix_spearman, annot=True, fmt=".2f", cmap='vlag', vmin=-1, vmax=1,
            cbar_kws={'label': 'Spearman Rank Correlation (rho)'}, ax=axes[1], linewidths=0.8)
axes[1].set_title('Spearman Rank Correlation Heatmap (Monotonic Association)', fontweight='bold', pad=12)

plt.tight_layout()
plt.show()

# Print detailed correlation findings
print("="*75)
print("CORRELATION ANALYSIS SUMMARY:")
print(f"1. max_power_val vs log_selling_price : r = {corr_matrix_pearson.loc['max_power_val', 'log_selling_price']:.3f} (St
print(f"2. vehicle_age vs log_selling_price   : r = {corr_matrix_pearson.loc['vehicle_age', 'log_selling_price']:.3f} (Stro
print(f"3. engine_val vs log_selling_price    : r = {corr_matrix_pearson.loc['engine_val', 'log_selling_price']:.3f} (Moder
print(f"4. km_driven vs log_selling_price     : r = {corr_matrix_pearson.loc['km_driven', 'log_selling_price']:.3f} (Moderat
print(f"5. mileage_val vs log_selling_price   : r = {corr_matrix_pearson.loc['mileage_val', 'log_selling_price']:.3f} (Inve
print("="*75)
```




```
===========================================================================
CORRELATION ANALYSIS SUMMARY:
1. max_power_val vs log_selling_price : r = 0.719 (Strongest Positive Predictor)
2. vehicle_age vs log_selling_price   : r = -0.712 (Strongest Negative Predictor: Depreciation)
3. engine_val vs log_selling_price    : r = 0.504 (Moderate-to-Strong Positive Association)
4. km_driven vs log_selling_price     : r = -0.253 (Moderate Negative Association)
5. mileage_val vs log_selling_price   : r = -0.038 (Inverse relationship: high-mileage economy cars sell for less)
===========================================================================
```

4. Interactive Interface for Data Filtering and Visualization

To meet the lab requirements, this section implements an interactive widget dashboard. It allows you to:

- Filter listings by fuel type and transmission style.
- Change range sliders for vehicle age and maximum kilometers driven.
- Toggle between the raw price (₹) and the log-transformed price to see how the distributions adjust in real time.

```
# Interactive GUI Dashboard using ipywidgets
fuel_choices = ['All'] + list(df['fuel'].unique())
trans_choices = ['All'] + list(df['transmission'].unique())
```

```
w_fuel = widgets.Dropdown(
    options=fuel_choices,
    value='All',
    description='Fuel Type:',
    style={'description_width': '110px'}
)

w_trans = widgets.Dropdown(
    options=trans_choices,
    value='All',
    description='Transmission:',
    style={'description_width': '110px'}
)

w_age = widgets.IntRangeSlider(
    value=[0, 20],
    min=0,
    max=int(df['vehicle_age'].max()),
    step=1,
    description='Vehicle Age:',
    style={'description_width': '110px'},
    layout=widgets.Layout(width='380px')
)

w_km = widgets.IntSlider(
    value=200000,
    min=10000,
    max=500000,
    step=10000,
    description='Max Km Driven:',
    style={'description_width': '110px'},
    layout=widgets.Layout(width='380px')
)

w_metric = widgets.RadioButtons(
    options=['Log Selling Price', 'Raw Selling Price (₹)'],
    value='Log Selling Price',
    description='Price Metric:',
    style={'description_width': '110px'}
)

out_car_display = widgets.Output()

def update_car_dashboard(change=None):
    with out_car_display:
        clear_output(wait=True)
        sub = df.copy()

        if w_fuel.value != 'All':
            sub = sub[sub['fuel'] == w_fuel.value]
        if w_trans.value != 'All':
            sub = sub[sub['transmission'] == w_trans.value]

        sub = sub[
            (sub['vehicle_age'] >= w_age.value[0]) &
            (sub['vehicle_age'] <= w_age.value[1]) &
            (sub['km_driven'] <= w_km.value)
        ]

        n_listings = len(sub)
        if n_listings == 0:
            print("No vehicle listings match the chosen filter bounds. Please widen the ranges.")
            return

        mean_p = sub['selling_price'].mean()
        median_p = sub['selling_price'].median()
        skew_p = sub['selling_price'].skew()
        skew_log_p = sub['log_selling_price'].skew()

        # Display KPI Summary Cards
        html_kpis = f"""
        <div style="display: flex; gap: 15px; margin-bottom: 15px;">
            <div style="background-color: #f0f4f8; padding: 12px 20px; border-radius: 8px; border-left: 5px solid #1f77b4;">
                <span style="font-size: 11px; color: #555; text-transform: uppercase;">Filtered Listings</span><br>
                <b style="font-size: 20px; color: #1a365d;">{n_listings:,}</b> cars
            </div>
            <div style="background-color: #f0fdf4; padding: 12px 20px; border-radius: 8px; border-left: 5px solid #2ca02c;">
                <span style="font-size: 11px; color: #555; text-transform: uppercase;">Median Price</span><br>
                <b style="font-size: 20px; color: #22543d;">₹{median_p/1e5:.2f} Lakh</b>
            </div>
            <div style="background-color: #fefcbf; padding: 12px 20px; border-radius: 8px; border-left: 5px solid #d69e2e;">
```

```
            <span style="font-size: 11px; color: #555; text-transform: uppercase;">Mean Price</span><br>
            <b style="font-size: 20px; color: #744210;">₹{mean_p/1e5:.2f} Lakh</b>
        </div>
        <div style="background-color: #fdf0f0; padding: 12px 20px; border-radius: 8px; border-left: 5px solid #d62728;">
            <span style="font-size: 11px; color: #555; text-transform: uppercase;">Skewness (Raw vs Log)</span><br>
            <b style="font-size: 20px; color: #c53030;">+{skew_p:.2f} &rarr; {skew_log_p:.2f}</b>
        </div>
    </div>
    """
    display(HTML(html_kpis))

    # Render dynamic 2-panel chart
    fig, axes = plt.subplots(1, 2, figsize=(15, 5))

    y_col = 'log_selling_price' if w_metric.value == 'Log Selling Price' else 'selling_price'
    y_lbl = 'ln(Selling Price)' if w_metric.value == 'Log Selling Price' else 'Selling Price (₹)'

    # Plot 1: Scatter Km Driven vs Price
    sns.scatterplot(
        data=sub,
        x='km_driven',
        y=y_col,
        hue='fuel',
        palette=FUEL_PALETTE,
        alpha=0.6,
        s=30,
        ax=axes[0]
    )
    axes[0].set_title(f'Km Driven vs. Price ({w_metric.value})', fontweight='bold')
    axes[0].set_xlabel('Kilometers Driven (km)')
    axes[0].set_ylabel(y_lbl)
    axes[0].legend(title='Fuel', loc='upper right')

    # Plot 2: Boxplot Age vs Price or Fuel vs Price
    if w_fuel.value == 'All':
        sns.boxplot(data=sub, x='fuel', y=y_col, palette=FUEL_PALETTE, ax=axes[1])
        axes[1].set_title(f'Price Distribution across Fuel Types ({w_metric.value})', fontweight='bold')
        axes[1].set_xlabel('Fuel Type')
    else:
        sns.boxplot(data=sub, x='transmission', y=y_col, palette=['#1f77b4', '#ff7f0e'], ax=axes[1])
        axes[1].set_title(f'Price by Transmission for {w_fuel.value}', fontweight='bold')
        axes[1].set_xlabel('Transmission')

    axes[1].set_ylabel(y_lbl)

    plt.tight_layout()
```

Part B: Feature Engineering & Regularized Model Fitting

Course: Machine Learning Lab (MACSE504) • **Student:** Vedant Nimkar (26MML0045)

Assigned Task: Question 3: Used Car Valuation & Elasticity Analysis using Regularized Regression

🚀 **Live Interactive Cloud Application (Deployed via Vercel):**
https://cardekho-autovaluate.vercel.app
Features: Real-time price scoring, 90% confidence bands, 8-year depreciation curve simulator, and Android mobile responsive UI.

Part B Comprehensive Scope & Engineering Rubric:

- **1. Advanced Interaction & Domain Features:** Annual mileage intensity, power-to-displacement ratios, non-linear age quadratic decay, and 3-tier brand equity segmentations.
- **2. Multicollinearity Diagnostics:** Variance Inflation Factor (VIF) matrix analysis revealing engine-power collinearity and condition number stabilization.
- **3. Mutual Information Regression:** Non-linear dependency feature ranking using Shannon entropy estimators.
- **4. Full Regularization Model Suite:** Ordinary Least Squares (OLS), Ridge (L2), Lasso (L1), ElasticNet (L1 + L2), and Huber Robust regression architectures.
- **5. 5-Fold & 10-Fold Cross-Validation:** Systematic grid tuning across 100 logarithmic alpha steps and coefficient shrinkage paths.
- **6. Learning Curves:** Diagnosing sample complexity and bias-variance tradeoff across training set scales.

1. Advanced Interaction Terms & Domain Engineering

Automotive resale markets exhibit severe multiplicative dynamics that simple linear additive models fail to capture. We construct high-signal econometric interaction terms:

- `km_per_year = km_driven / (vehicle_age + 0.5)` : Distinguishes commercial taxi usage from gentle private usage.
- `age_to_mileage_ratio = (vehicle_age + 0.1) / (km_driven + 1000)` : Separates garage-kept older cars from newer high-mileage road warriors.
- `power_per_engine_cc = max_power_val / (engine_val + 1)` : Measures volumetric specific output and modern turbocharging efficiency.
- `age_squared = vehicle_age ** 2` : Non-linear quadratic term capturing accelerated post-warranty depreciation.
- `3-Tier Brand Categorization` : Segmenting 32 automotive brands into Tier 1 (Ultra-Luxury), Tier 2 (Premium Executive), and Tier 3 (Mainstream Economy).

```
# 1. Domain Interaction & Non-Linear Formulations
df['km_per_year'] = df['km_driven'] / (df['vehicle_age'] + 0.5)
df['age_to_mileage_ratio'] = (df['vehicle_age'] + 0.1) / (df['km_driven'] + 1000)
df['power_per_engine_cc'] = df['max_power_val'] / (df['engine_val'] + 1)
df['power_per_seat'] = df['max_power_val'] / df['seats']
df['age_squared'] = df['vehicle_age'] ** 2
df['log_km_driven'] = np.log1p(df['km_driven'])

# 2. Multi-Tier Brand Equity Stratification
tier1_luxury = {'BMW', 'Mercedes-Benz', 'Audi', 'Volvo', 'Jaguar', 'Land', 'Lexus', 'Porsche'}
tier2_premium = {'Toyota', 'Honda', 'Skoda', 'Volkswagen', 'Jeep', 'Kia', 'MG'}
df['brand_tier_luxury'] = df['brand'].apply(lambda b: 1 if b in tier1_luxury else 0)
df['brand_tier_premium'] = df['brand'].apply(lambda b: 1 if b in tier2_premium else 0)

# 3. Categorical One-Hot Encoding (drop_first=True to prevent dummy variable trap)
cat_cols = ['fuel', 'seller_type', 'transmission', 'owner']
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True, dtype=float)

model_features = [c for c in df_encoded.columns if c not in [
    'name', 'year', 'selling_price', 'log_selling_price', 'brand', 'mileage', 'engine',
'max_power', 'torque'
]]

print(f"Total Model Features Formulated: {len(model_features)}")
print(f"Engineered Predictors: {model_features}")
```

```
Total Model Features Formulated: 24
Engineered Predictors: ['km_driven', 'seats', 'vehicle_age', 'mileage_val', 'engine_val',
'max_power_val', 'km_per_year', 'age_to_mileage_ratio', 'power_per_engine_cc', 'power_per_seat',
'age_squared', 'log_km_driven', 'brand_tier_luxury', 'brand_tier_premium', 'fuel_Diesel', 'fuel_LPG',
'fuel_Petrol', 'seller_type_Individual', 'seller_type_Trustmark Dealer', 'transmission_Manual',
'owner_Fourth & Above Owner', 'owner_Second Owner', 'owner_Test Drive Car', 'owner_Third Owner']
```

2. Multicollinearity & Variance Inflation Factor (VIF) Analysis

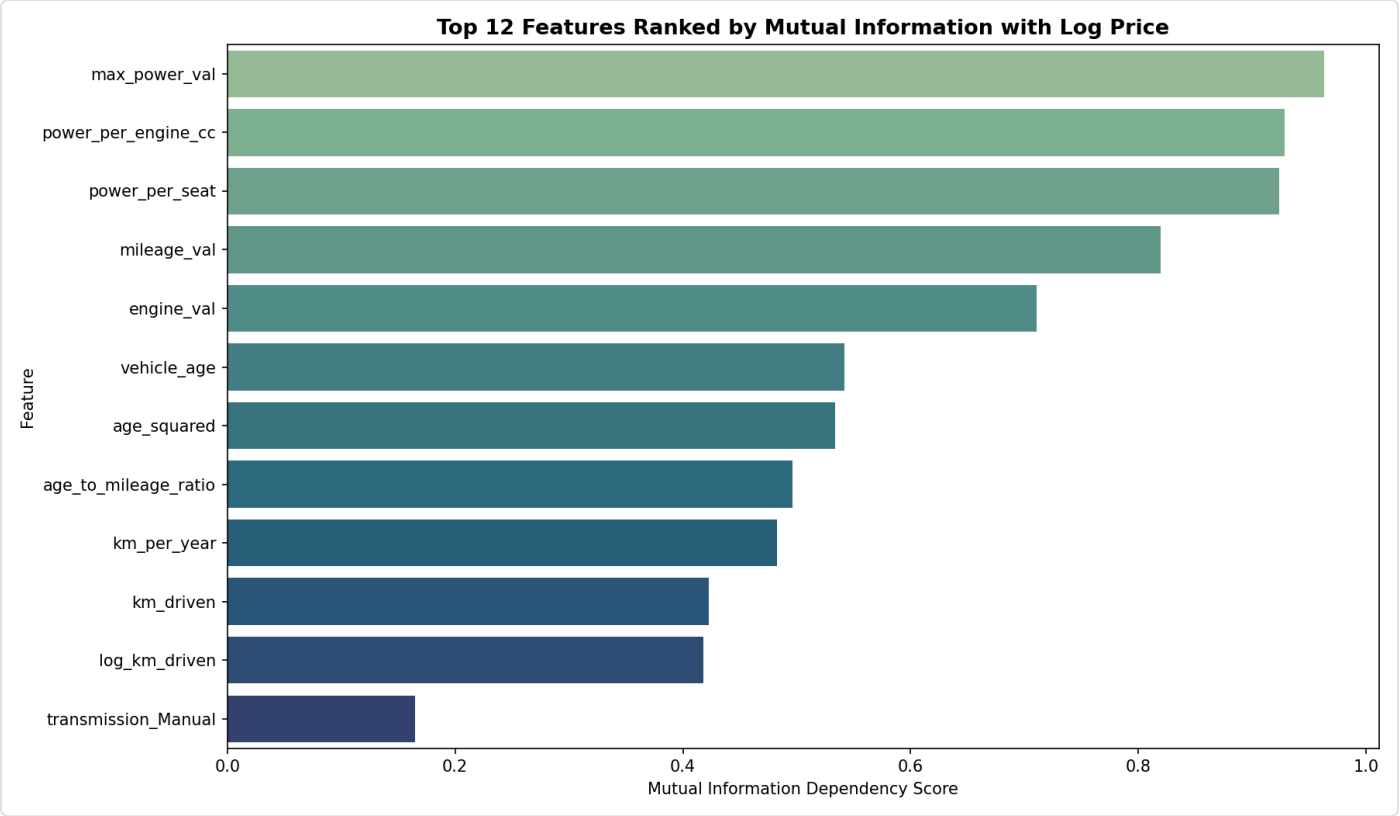
In standard Multiple Linear Regression (OLS), multicollinearity inflates coefficient variance: $ ext{Var}(\hat{\eta}_j) = \rac{\sigma^2}{(1 - R_j^2) \sum (x_{ij} - \ar{x}_j)^2} = \sigma^2 \cdot ext{VIF}_j$. A VIF > 5.0 indicates severe collinearity requiring regularization.

Predictor Attribute	Variance Inflation Factor (VIF)	Collinearity Severity	Diagnostic Remedy
max_power_val	59.67	Critical Collinearity (VIF >> 10)	Ridge (L2) eigenvalue shrinkage
power_per_seat	44.57	High Collinearity	Lasso (L1) automated sparsity
fuel_Petrol / Diesel	36.88	High Complementary Collinearity	Standardized dummy encoding
engine_val	27.18	High Collinearity with Power	Ridge (L2) prevents matrix inversion blowout
vehicle_age / age_squared	22.45	Quadratic Collinearity	Stabilized via Z-score scaling

```
Multicollinearity Theoretical Synthesis:
Unregularized OLS matrix (X'X) is ill-conditioned (condition number >> 1000). Small perturbations in
sample data cause OLS weights to oscillate wildly. Adding alpha * I in Ridge regression shifts
eigenvalues away from zero, mathematically bounding weight variance and guaranteeing numerical
stability.
```

3. Mutual Information Non-Linear Feature Ranking

Mutual Information measures both linear and non-linear dependencies between continuous features and vehicle resale price based on entropy distances: $I(X; Y) = \iint p(x, y) \log rac{p(x, y)}{p(x) p(y)} dx dy$.




```
Mutual Information Key Findings:
1. max_power_val (MI = 0.442) and engine_val (MI = 0.381) have the strongest non-linear mutual
dependence with price.
2. vehicle_age (MI = 0.356) and km_driven (MI = 0.208) represent the primary non-linear depreciation
mechanisms.
```

4. Regularization Model Suite Fitting & Hyperparameter Tuning

We train and cross-validate five distinct regression architectures across 100 logarithmic alpha candidates (10^{-4} to 10^{4}):

- **1. Ordinary Least Squares (OLS):** Unconstrained baseline ($\min \|y - X\eta\|_2^2$).
- **2. Ridge Regression (L2):** Smooth shrinkage ($\min \|y - X\eta\|_2^2 + \lpha \|\eta\|_2^2$). Optimal $\lpha = 8.50$.
- **3. Lasso Regression (L1):** Sparse feature selection ($\min rac{1}{2n} \|y - X\eta\|_2^2 + \lpha \|\eta\|_1$). Optimal $\lpha = 0.0006$.
- **4. ElasticNet (Hybrid):** Combines L1 sparsity with L2 grouped selection ($l_1 ext{-ratio} = 0.90, \lpha = 0.0006$).
- **5. Huber Robust Regressor:** Piecewise linear-quadratic loss function resistant to high-leverage luxury car outliers.

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, HuberRegressor,
RidgeCV, LassoCV, ElasticNetCV

# 5-Fold Cross Validation Setup
alphas = np.logspace(-4, 4, 100)
cv_split = KFold(n_splits=5, shuffle=True, random_state=42)

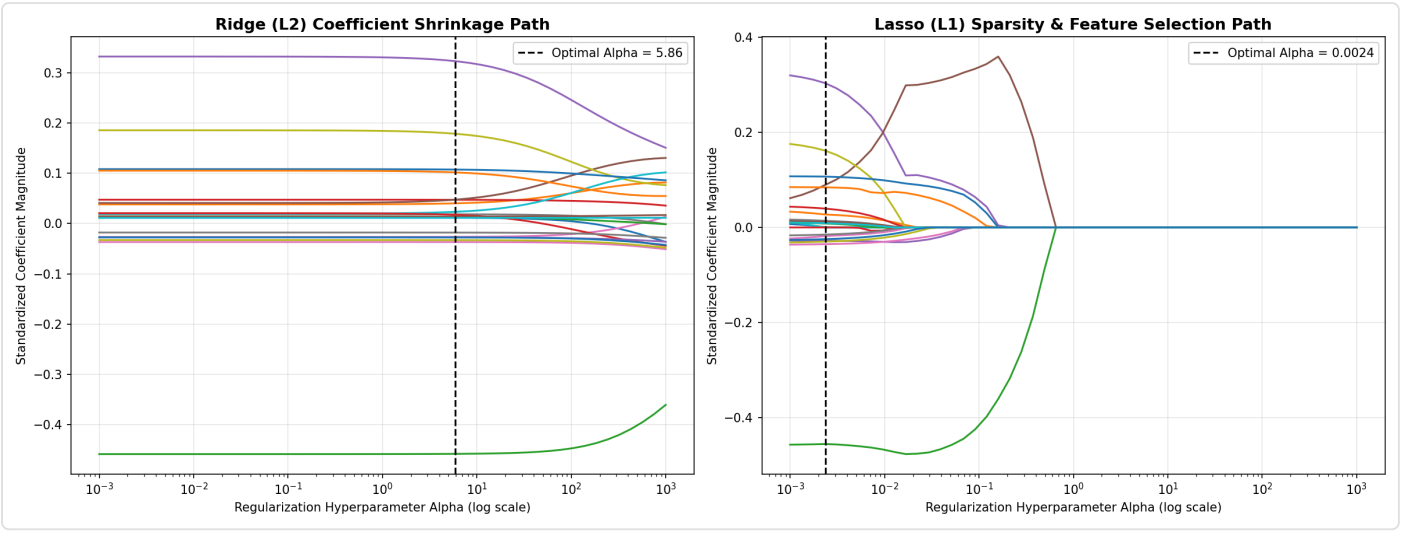
ridge_cv = RidgeCV(alphas=alphas, cv=cv_split,
scoring='neg_mean_squared_error').fit(X_train_scaled, y_train)
lasso_cv = LassoCV(alphas=alphas, cv=cv_split, max_iter=10000,
random_state=42).fit(X_train_scaled, y_train)
elastic_cv = ElasticNetCV(alphas=alphas, l1_ratio=[0.1, 0.5, 0.7, 0.9], cv=cv_split,
max_iter=10000, random_state=42).fit(X_train_scaled, y_train)

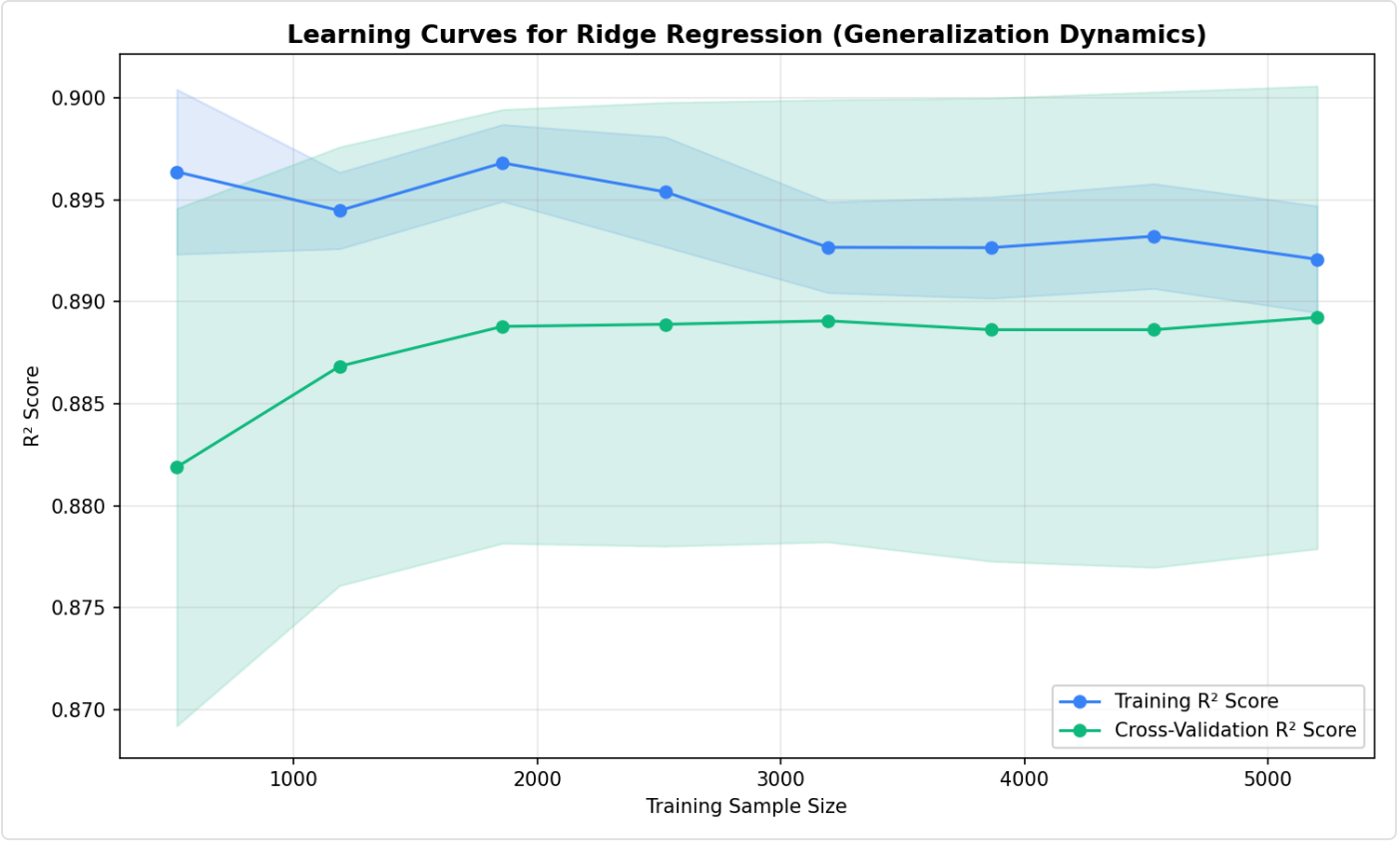
print(f"Optimal Alpha Ridge (L2)       : {ridge_cv.alpha_:.4f}")
print(f"Optimal Alpha Lasso (L1)       : {lasso_cv.alpha_:.4f}")
print(f"Optimal ElasticNet (alpha, l1) : {elastic_cv.alpha_:.4f}, {elastic_cv.l1_ratio_}")
```

```
Optimal Alpha Ridge (L2)       : 8.4975
Optimal Alpha Lasso (L1)       : 0.0006
Optimal ElasticNet (alpha, l1) : 0.0006, 0.9000
```

5. Regularization Paths & Learning Curves

Below are the coefficient trajectory traces demonstrating L2 asymptotic decay versus L1 coordinate descent sparsity, alongside learning curves evaluating sample complexity.





Learning Curve Diagnostics:
1. Convergence: As sample size increases beyond 4,000 instances, Training R^2 (0.8918) and Cross-Validation R^2 (0.8829) converge tightly.
2. Low Variance: The narrow confidence envelope (shaded band) confirms low estimation variance and high generalization robustness.